GUIDE 59 · GOVERN

Verify a release before you run it

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=59-verify-a-release-before-you-run-it. This copy was made on 4 November 2026.

Time: 20 minutes. **You'll end up with:** proof that the image and chart you run were built by this repository's release workflow, from the commit it names - and a cluster that refuses anything else.

From 0.31, every image and the Helm chart is signed keyless with cosign, by digest, by `.github/workflows/docker.yml` on the release's tag, with SLSA provenance, an SBOM and a GitHub attestation (ADR 0018). Keyless means there's no key to trust: the signing certificate names the workflow and the tag it ran on, and the signature is recorded in Sigstore's public log.

Every command and output below was run for real, against the release workflow's **rehearsal** - the same workflow, run by hand on `main`, publishing under `-rehearsal` names (`ghcr.io/anantharajuc/queryapigate-rehearsal`). So its identity ends `@refs/heads/main`. For a release, it ends `@refs/tags/v<X.Y.Z>`, and the names have no `-rehearsal`:

	REHEARSAL (BELOW)	A RELEASE
Image	`ghcr.io/anantharajuc/queryapigate-rehearsal`	`ghcr.io/anantharajuc/queryapigate`
Chart	`ghcr.io/anantharajuc/charts-rehearsal/queryapigate`	`ghcr.io/anantharajuc/charts/queryapigate`
Identity	`https://github.com/AnanthaRajuC/QueryAPIGate/.github/workflows/docker.yml@refs/heads/main`	`...docker.yml@refs/tags/v0.31.0`

The tools: cosign 3.1.3 and, for GitHub's attestation, the `gh` CLI (or cosign, as shown).

Check a signature

```shell
cosign verify ghcr.io/anantharajuc/queryapigate-rehearsal:rehearsal-61cfb4fc727d \
  --certificate-identity
https://github.com/AnanthaRajuC/QueryAPIGate/.github/workflows/docker.yml@refs/heads/main \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

```text
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates
```

The same for the `-h2` image and the chart. What doesn't verify:

```text
# the same image, as the v0.31.0 release - it was signed on main, not that tag
Error: no matching attestations: failed to verify certificate identity: no matching CertificateIdentity
found ...

# as another repository's workflow
Error: no matching attestations: failed to verify certificate identity: no matching CertificateIdentity
found ...

# 0.30.0, released before signing
Error: no signatures found
```

Each release is signed in two forms: the Sigstore bundle (`cosign verify`, the default) and the classic `.sig` tag (`cosign verify --new-bundle-format=false`). Both verify; admission controllers below need the second.

See how it was built, and from what

GitHub's attestation names the repository, the workflow, the commit and the run:

```shell
cosign verify-attestation ghcr.io/anantharajuc/queryapigate-
rehearsal@sha256:6e4560b1228386bcf637015eea3f729e015f47267aa6b5ad1c88d682bd82bd4f \
  --new-bundle-format --type https://slsa.dev/provenance/v1 \
  --certificate-identity
https://github.com/AnanthaRajuC/QueryAPIGate/.github/workflows/docker.yml@refs/heads/main \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

```text
predicate https://slsa.dev/provenance/v1
  workflow https://github.com/AnanthaRajuC/QueryAPIGate .github/workflows/docker.yml refs/heads/main
  commit git+https://github.com/AnanthaRajuC/QueryAPIGate@refs/heads/main {'gitCommit':
'61cfb4fc727d64812add0c8f7aef92b36e58ee28'}
  run https://github.com/AnanthaRajuC/QueryAPIGate/actions/runs/41208623399/attempts/1
```

(decoded from the attestation's payload). With the `gh` CLI signed in, `gh attestation verify oci://<image> --owner AnanthaRajuC` checks the same. The image's own provenance, from the build, records what it was built on - the base images by digest, for `linux/amd64` (the `&platform=` suffix left off):

```shell
docker buildx imagetools inspect ghcr.io/anantharajuc/queryapigate-rehearsal:rehearsal-61cfb4fc727d --
format '{{json .Provenance}}'
```

```text
pkg:docker/debian@trixie-slim?
digest=sha256:f6b08b09a05725f9431502d3b5714f5c4576e82c1a84fa1debe0c24a7e0c3ec4
pkg:docker/node@22-slim?digest=sha256:de170b39439eb0e5a3444ad7d60b015614a381e10fecae2e30ec9e735122f38d
pkg:docker/rust@1.98.1-slim-trixie?
digest=sha256:4cd829461bd5c4d511c32e269da9cb8929223b666519d8004e35fc8d1d771ab7
pkg:docker/ghcr.io/astral-sh/uv@0.13.6?
digest=sha256:af22a52f5eafef3d78e551599d405c02df61919d18028f0cea8eda93bc9986d0
```

The `uv` image is Python's pin: `uv` installs the image's Python and checks its hash, and each `uv` release resolves `3.13` to one patch release - 0.13.6 to 3.13.18 (ADR 0031). The Rust image is `linux/amd64` for both platforms, because the data plane is cross-compiled (ADR 0030).

Read the SBOM

```shell
docker buildx imagetools inspect ghcr.io/anantharajuc/queryapigate-rehearsal:rehearsal-61cfb4fc727d --
format '{{json .SBOM}}'
```

An SPDX 2.3 document per platform - 383 packages for `linux/amd64` (191 of them the data plane's Rust crates), among them:

```shell
queryapigate 0.47.0
python 3.13.18
flask 3.1.3
werkzeug 3.1.9
cryptography 50.0.2
openssl 3.5.7-1~deb13u3
```

(a rehearsal on `main` carries the last release's version). `python` is the interpreter itself, found at `/opt/python/bin/python3.13` and named by CPE, so a scanner reading the SBOM matches Python's own advisories to it.

Feed it to your scanner (Grype, Trivy) to see which advisories apply, without pulling the image.

Have a cluster refuse anything else

Sigstore policy-controller

On a kind cluster with policy-controller 0.13.1 (Helm chart `sigstore/policy-controller` 0.10.8), a policy admitting only images this workflow signed on a release tag:

```yaml
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: queryapigate-signed
spec:
  images:
    - glob: "ghcr.io/anantharajuc/**"
  authorities:
    - keyless:
        url: https://fulcio.sigstore.dev
        identities:
          - issuer: https://token.actions.githubusercontent.com
            subjectRegExp:
'^https://github\.com/AnanthaRajuC/QueryAPIGate/\.github/workflows/docker\.yml@refs/tags/v[0-9]+\.[0-
9]+\.[0-9]+$'
      ctlog:
        url: https://rekor.sigstore.dev
```

and `kubectl label namespace qag policy.sigstore.dev/include=true`. With the identity widened to `refs/heads/main` too, the rehearsal image ran (`pod/signed created`, then `/health` answered `0.30.0`); without, or for 0.30.0:

```text
admission webhook "policy.sigstore.dev" denied the request: ... signature keyless validation failed for
authority
authority-0 for ghcr.io/anantharajuc/queryapigate-rehearsal@sha256:41a3...: no matching signatures: none
of the
expected identities matched ...

admission webhook "policy.sigstore.dev" denied the request: ...
ghcr.io/anantharajuc/queryapigate:0.30.0@sha256:4fca...
signature keyless validation failed ...: no signatures found
```

policy-controller finds only the classic `.sig` form - an image signed with a Sigstore bundle alone is refused as unsigned, which is why releases carry both.

Kyverno

With Kyverno 1.19.1 (Helm chart `kyverno/kyverno` 3.9.1):

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: queryapigate-signed
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 30
  rules:
    - name: signed-by-its-release-workflow
      match:
        any:
          - resources:
              kinds: [Pod]
      verifyImages:
        - imageReferences: ["ghcr.io/anantharajuc/*"]
          mutateDigest: true
          verifyDigest: true
          attestors:
            - entries:
                - keyless:
                    issuer: https://token.actions.githubusercontent.com
                    subjectRegExp:
'^https://github\.com/AnanthaRajuC/QueryAPIGate/\.github/workflows/docker\.yml@refs/tags/v[0-9]+\.[0-
9]+\.[0-9]+$'
                    roots: |
                      <the certificates from https://fulcio.sigstore.dev/api/v2/trustBundle>
                    rekor:
                      url: https://rekor.sigstore.dev
```

Give it `roots`. Without them Kyverno 1.19.1 refused the signed image - `cert verification failed: x509: certificate signed by unknown authority` - because its built-in Sigstore root predates Fulcio's current intermediate. With the two certificates Fulcio publishes at `/api/v2/trustBundle`, it admitted the signed image and rewrote the pod to run it by digest (`mutateDigest`), and refused 0.30.0: `no signatures found`.

On the Helm chart

Verify the chart before installing it, and install by digest:

```shell
cosign verify ghcr.io/anantharajuc/charts/queryapigate:0.31.0 \
  --certificate-identity
https://github.com/AnanthaRajuC/QueryAPIGate/.github/workflows/docker.yml@refs/tags/v0.31.0 \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
helm install qag oci://ghcr.io/anantharajuc/charts/queryapigate --version 0.31.0 \
  --set image.digest=sha256:<the image's digest, from cosign verify>
```

Good to know

- **A signature proves who built it, not that it's right**: this repository's workflow, on that tag, from that commit. A maintainer account that's compromised can still tag a release - and Rekor's public log then records it.
- **Pin by digest, not tag.** A tag can be moved to other bytes; a digest can't, and it's what was signed.

- **Releases before 0.31 aren't signed.** A policy like the ones above refuses them.

Next steps

- SECURITY.md - the two commands, for a release.
- ADR 0018 - what's signed, how, and what it doesn't promise.
- Install on Kubernetes with Helm.